

Imprecise Probabilistic Programming, Precisely (Functional Pearl)

Credal Sets via Graded Monads, BDDs, and Semiring-Parametric Inference

Jack Liell-Cock Sam Staton

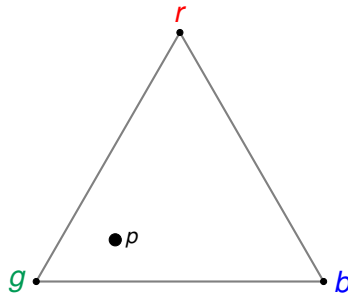
University of Oxford

ICFP 2026

Imprecise probability: from one distribution to many

Probability

- A point p in the simplex



The simplex over $\{r, g, b\}$

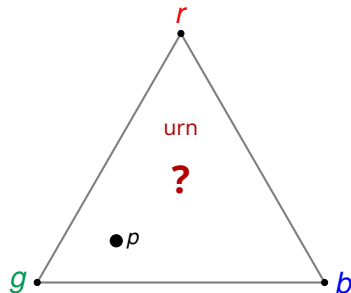
Imprecise probability: from one distribution to many

Probability

- A point p in the simplex

Ellsberg's urn

- **30 red** balls, **60 green or blue** balls
- People prefer to bet on red over green
- No point justifies this preference



The simplex over $\{r, g, b\}$

Imprecise probability: from one distribution to many

Probability

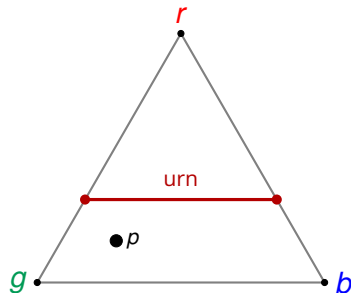
- A point p in the simplex

Ellsberg's urn

- **30 red** balls, **60 green or blue** balls
- People prefer to bet on red over green
- No point justifies this preference

Imprecise probability

- A convex set of points: a *credal set*
- *Knightian* uncertainty: no point is privileged
- The urn: $P(r) = \frac{1}{3}$ but $P(g), P(b) \in [0, \frac{2}{3}]$
- Inference targets *min* and *max* probabilities



The simplex over $\{r, g, b\}$

Contributions

1. A Haskell-embedded **graded-monad DSL** for an imprecise PPL.
Type-level names [Lew et al., 2020] track each source of Knightian uncertainty, building on the theory in [Liell-Cock and Staton, 2025].
2. Imprecise inference needs no new compilation infrastructure.
Probabilistic *and* Knightian choices compile to BDD variables.
3. One compilation, **four** semiring-based inference methods.
Exact enumeration, gradient descent, symbolic multinomials, and interval bounds.

Core is $\approx$ 1,200 lines of Haskell. Standard BDD + WMC machinery.

Package at <https://hackage.haskell.org/package/imp-ppl>

Implementation at github.com/jacklc3/imp

The graded monad, in Haskell

$\text{Imp } g \ a$: an imprecise computation over a with Knightian inputs g

```
data Ball = Red | Green | Blue

ellsberg :: Imp '["split"] Ball
ellsberg = Imp.do
  isRed    ← flip (1/3)
  isGreen  ← knight @"split"
  Imp.return $
    if isRed then Red
    else if isGreen then Green
    else Blue
```

```
ghci> intervalProbability ellsberg (== Green)
(0.0, 0.6667)
```

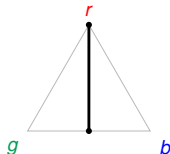
- `flip p :: Imp '[] Bool`
a biased coin
- `knight @"n" :: Imp '["n"] Bool`
a coin of *unknown* bias
- `QualifiedDo` for ergonomic do-notation
- **bind** unions names, enforcing disjointness
- Names **erased at runtime**

Semantically, $\text{Imp } g \ a = 2^g \rightarrow D(a)$. A distribution for each Knightian valuation.

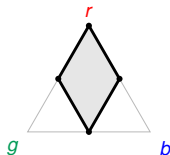
Names dictate dependence

Each name is **one unknown bias**: same name = dependent; different names = independent.

```
dependent :: Imp '["a1"] Ball
dependent = Imp.do
  x ← flip 0.5
  if x then Imp.do
    y ← knight @"a1"
    Imp.return (if y then Red else Green)
  else Imp.do
    y ← knight @"a1"
    Imp.return (if y then Red else Blue)
```



```
independent :: Imp '["a1", "a2"] Ball
independent = Imp.do
  x ← flip 0.5
  if x then Imp.do
    y ← knight @"a1"
    Imp.return (if y then Red else Green)
  else Imp.do
    y ← knight @"a2" -- a2, not a1
    Imp.return (if y then Red else Blue)
```



Names restore commutativity

The evaluation *order* of two independent programs should not matter.

<code>do</code>	<code>x</code>	<code>←</code>	<code>p</code>		<code>do</code>	<code>y</code>	<code>←</code>	<code>q</code>
	<code>y</code>	<code>←</code>	<code>q</code>	<code>=</code>		<code>x</code>	<code>←</code>	<code>p</code>
	<code>f</code>	<code>x</code>	<code>y</code>			<code>f</code>	<code>x</code>	<code>y</code>

The convex powerset monad (the standard monad for credal sets) breaks this: dependency of Knightian choices is not tracked, so reordering is unsound.

Names restore commutativity

The evaluation *order* of two independent programs should not matter.

$$\begin{array}{l} \text{do } x \leftarrow p \\ \quad y \leftarrow q \\ \quad f \ x \ y \end{array} = \begin{array}{l} \text{do } y \leftarrow q \\ \quad x \leftarrow p \\ \quad f \ x \ y \end{array}$$

Grading by **names** restores commutativity whenever the Knightian names of p and q are **disjoint** [Liell-Cock and Staton, 2025]

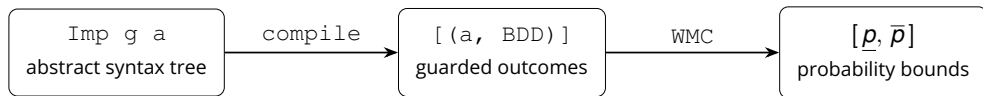
Disjointness is checked by GHC

A type-level disjoint union of the name sets rejects reuse across binds:

```
error: Duplicate Knightian name: "a1"
```

Explicit renaming via `tag` is available when independence is intended.

The BDD pipeline



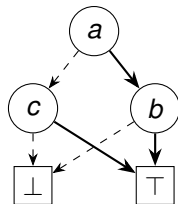
A BDD is a canonical DAG for a Boolean formula.

The same pipeline as *precise* discrete languages (e.g. Dice [Holtzen et al., 2020]).

Compilation by structural recursion:

- `flip p`: a fresh *weighted* BDD variable (weight p)
- `knight @"n"`: an *unweighted* BDD variable; same name = same variable

if a then b else c



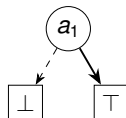
A Knightian coin flip is just a BDD variable whose weight is left free.

Names in the compiled BDD

Program

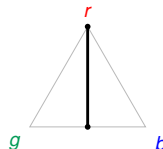
```
dependent :: Imp '["a1"] Ball
dependent = Imp.do
  x ← flip 0.5
  if x then Imp.do
    y ← knight @"a1"
    Imp.return (if y then Red else Green)
  else Imp.do
    y ← knight @"a1"
    Imp.return (if y then Red else Blue)
```

BDD (for Red)

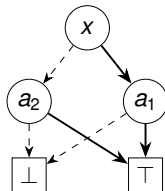


shared name: collapses

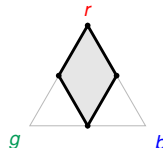
Credal set



```
independent :: Imp '["a1", "a2"] Ball
independent = Imp.do
  x ← flip 0.5
  if x then Imp.do
    y ← knight @"a1"
    Imp.return (if y then Red else Green)
  else Imp.do
    y ← knight @"a2" -- a2, not a1
    Imp.return (if y then Red else Blue)
```



distinct names: stay separate



One WMC, four semirings

WMC only uses $(0, 1, +, \cdot)$, so it runs over **any semiring** [Kimmig et al., 2017]:

$$\text{WMC}(\top) = 1, \quad \text{WMC}(\perp) = 0, \quad \text{WMC}(v \mapsto (l, h)) = w_v^\perp \cdot \text{WMC}(l) + w_v^\top \cdot \text{WMC}(h)$$

	Real	Gradient	Symbolic	Interval
Probabilistic weight	p	$(p, \vec{0})$	p	$[p, p]$
Knightian weight	0 or 1	$(0.5, \vec{\delta}_n)$	x_n	$[0, 1]$
Computes	exact bounds	gradients	multinomial	outer bound
Cost (k knights)	2^k passes	passes $\propto$ steps	1 pass + 2^k corners	1 pass

WMC is *multilinear* in the weights: extrema sit at Knightian values 0 or 1.

Gradient WMC as in Pluck [Bowers et al., 2025].

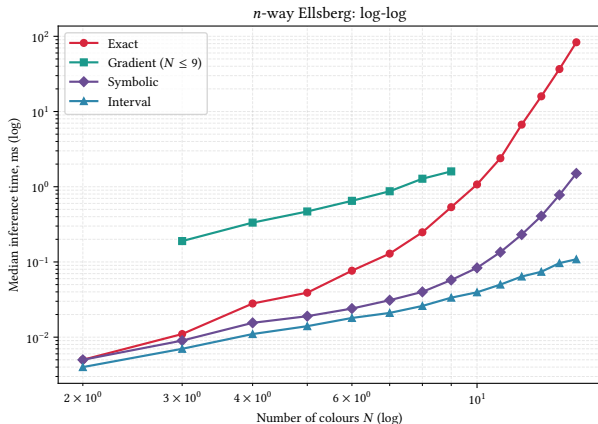
Performance on N -way Ellsberg urn

N -way Ellsberg urn:

- 1 ball has probability $\frac{1}{N}$
- Remaining have probability $[0, \frac{N-1}{N}]$
- Program is a BDD of size $O(N)$

Inference method:

- **Exact:** 2^{N-2} WMC passes
- **Gradient:** fixed number of passes
- **Symbolic:** single pass with *cheaper* 2^{N-2} corner search
- **Interval:** single pass



(Symbolic semiring added since the camera-ready.)

A standard pipeline

- Type-level **names** track dependence and restore commutativity: checked by GHC, erased at runtime
- Probabilistic *and* Knightian coins both compile to **BDD variables**: a knight is just a variable with a **free weight**
- One **semiring-parametric WMC** gives *four* inference methods

Imprecise probability requires no new compilation infrastructure.

Open directions: continuous distributions; SDDs / variable-order heuristics; iteration and infinite-dimensional Knightian structure.

See paper for more examples: conditioning, imprecise Monty Hall problem, interval-valued MDPs.

Package at <https://hackage.haskell.org/package/imp-ppl>

Implementation at github.com/jacklc3/imp

- Maddy Bowers, Alexander Lew, Joshua Tenenbaum, Armando Solar-Lezama, and Vikash Mansinghka. Stochastic lazy knowledge compilation for inference in discrete probabilistic programs. In *Proc. PLDI '25*, 2025. doi: 10.1145/3729325.
- Steven Holtzen, Guy Van den Broeck, and Todd Millstein. Scaling exact inference for discrete probabilistic programs. In *Proc. OOPSLA '20*, 2020. doi: 10.1145/3428208.
- Angelika Kimmig, Guy Van den Broeck, and Luc De Raedt. Algebraic model counting. *Journal of Applied Logic*, 2017. doi: 10.1016/j.jal.2016.11.031.
- Alexander Lew, Marco Cusumano-Towner, Benjamin Sherman, Michael Carbin, and Vikash Mansinghka. Trace types and denotational semantics for sound programmable inference in probabilistic languages. In *Proc. POPL '20*, 2020. doi: 10.1145/3371087.
- Jack Liell-Cock and Sam Staton. Compositional imprecise probability: A solution from graded monads and Markov categories. In *Proc. POPL '25*, 2025. doi: 10.1145/3704890.